Building a Mac-Android CLI Tool

https://gemini.google.com/app/034ad5dc1dc62597

User prompt: i wantg to build a cli tool for mac that include screen capture (images , video) and controls and many interesting simple features to conect to my android phone from my mac, even a way to watch the screen like remote desktop, any any more features that would work smoothely over usb or wireles conectivity

Response: Building a custom macOS CLI tool to bridge your Mac and Android phone is a fantastic project. Instead of writing video decoding pipelines and low-level USB drivers from scratch, the smartest approach for a robust CLI tool is to wrap and orchestrate battle-tested tools like **ADB (Android Debug Bridge)** and **Scrcpy** under a unified command structure (using Node.js/TypeScript, Python, Go, or Rust).

An architecture blueprint, feature roadmap, and code pattern to build your macOS CLI tool smoothly over both **USB** and **Wireless (Wi-Fi)** are outlined below.

Phase 1: Core Architecture & Tech Stack

- **Language Choice: Go** or **Python** or **Node.js** (Go is exceptional for single-binary macOS CLI tools with fast concurrent execution; Python or Node.js works equally well if you prefer rapid scripting).
- **Under-the-Hood Engine:**
  - `adb` (Android Debug Bridge): Handles device discovery, shell commands, file transfers, app management, and Wi-Fi port forwarding.
  - `scrcpy` binary: Powers the ultra-low-latency real-time screen-mirroring and remote desktop window view.

Phase 2: Feature Roadmap for Your CLI Tool

You can structure your CLI commands logically (e.g., `droid <command> [options]`):

1. **Remote Desktop / Screen Mirroring**
   - `droid mirror`: Launches a high-performance live window of your phone screen on your Mac.
   - *Under the hood:* Calls `scrcpy` with configurable flags (bitrate, max-size, window bounds).
2. **Screen Capture & Recording**
   - `droid screenshot --output ~/Desktop/shot.png`: Grabs an instant image frame.
   - `droid record --duration 10 --output ~/Desktop/video.mp4`: Records a video clip directly from the device storage or streams it to disk.
3. **Connectivity Control (USB & Wireless)**
   - `droid connect --wifi`: Automatically switches a USB-connected device to TCP/IP mode (`adb tcpip 5555`), grabs its local IP, disconnects the cable, and establishes a wireless bridge.
   - `droid devices`: Lists active devices with connection types (USB vs. Wi-Fi IP address) and battery status.
4. **Productivity & Extra Controls**
   - `droid clipboard --push`: Syncs your Mac's clipboard text directly to the Android clipboard.
   - `droid install app.apk`: Quick sideloading utility.
   - `droid audio`: Forwards device audio to your Mac (supported natively in modern scrcpy versions).
   - `droid webcam`: Turns your phone's high-res camera into a Mac webcam via USB/Wi-Fi.

Phase 3: Implementation Blueprint (Python / Node Example)

Below is a conceptual script layout showing how easy it is to orchestrate ADB and Scrcpy inside a custom CLI script.

1. Checking Prerequisites

Your CLI tool should verify if `adb` and `scrcpy` are installed on the Mac (and offer to install them via Homebrew if missing):

```
# Check dependencies
which adb >/dev/null || brew install android-platform-tools
which scrcpy >/dev/null || brew install scrcpy
```

2. Handling USB vs. Wireless Switching Logic

To switch smoothly from a wired USB connection to a wireless connection through your CLI:

```
import subprocess
import sys

def run_cmd(command):
    result = subprocess.run(command, shell=True, capture_output=True, text=True)
    return result.stdout.strip()

def setup_wireless():
    print("-> Detecting connected USB device...")
    devices = run_cmd("adb devices")
    if "device" not in devices.splitlines()[-1]:
        print("Error: No device found via USB. Plug in your phone first.")
```

```
        sys.exit(1)

    print("-> Setting up TCP/IP port 5555...")
    run_cmd("adb tcpip 5555")

    print("-> Fetching device IP address...")
    # Grab IP address dynamically via adb shell
    ip_address = run_cmd("adb shell ip route | awk '{print $9}'").split()[-1]

    if not ip_address:
        # Fallback method to get IP
        ip_address = run_cmd("adb shell am get-config | grep ...") # or netcfg

    print(f"-> Connecting wirelessly to {ip_address}:5555...")
    connect_output = run_cmd(f"adb connect {ip_address}:5555")
    print(connect_output)
    print("Success! You can now unplug your USB cable. Run your mirror command.")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "wifi":
        setup_wireless()
```

3. Triggering Screen Capture & Remote View

- **For a Screenshot:**

```
run_cmd("adb exec-out screencap -p > ~/Desktop/android_screen.png")
```

- **For Remote Desktop Mirroring:**

```
# Launches scrcpy with custom performance flags
subprocess.run(["scrcpy", "--max-size=1024", "--bit-rate=4M", "--window-title=Android Remote"])
```

Phase 4: Unique "Bonus" Features to Add

To make your tool stand out beyond standard screen mirroring, consider adding these utility features:

- **`droid focus` (Presentation Mode):** Automatically turns off the physical phone screen (`scrcpy --turn-screen-off`) while keeping the interactive window active on your Mac to save battery during long casting sessions.
- **`droid notify`:** Pulls and prints recent Android notification logs right inside your Mac terminal using `adb shell dumpsys notification`.
- **`droid open <url>`:** Instantly forces the Android device to open a specific link in its default mobile browser.

Getting Started

1. Enable **Developer Options** and **USB Debugging** on your Android device.
2. Choose your preferred language (Python or Go are great for managing system sub-processes).
3. Build a wrapper CLI that sequences `adb` commands for state handling and `scrcpy` for the video stream display.

Sources

1. Android Mirroring with Scrcpy - Dimas Maulana
2. Scrcpy - Display and Control Your Android Device via Linux Desktop
3. Scrcpy : How to mirror Android mobile screen on Macbook M1 [2022

User prompt: i want this to have nice tui as well

Response: Adding a **Terminal User Interface (TUI)** to your bridge tool takes it from a standard command-line script to an interactive dashboard. Instead of remembering flags, you get an interactive menu where you can view your connected devices, toggle screen mirroring with a keystroke, grab screenshots, and monitor battery or CPU stats in real time.

Phase 1: Choosing Your TUI Tech Stack

Since you are writing this tool for macOS, your tech stack should be lightweight, single-binary friendly, and snappy:

- **Go:** Use **`charmbracelet/bubbletea`** (the gold standard for modern, beautiful Go TUIs with Elm architecture) or **`gocui`**.
- **Python:** Use **`Textual`** (a modern async TUI framework for Python with CSS-like styling) or `Rich` + `Blessings`.
- **Node.js:** Use **`Ink`** (React for the terminal).

Phase 2: Core TUI Layout & Features

Your TUI can be structured with a split-pane or tabbed layout:

1. **Device Panel (Sidebar / Top Pane):** Lists connected Android devices via USB or Wi-Fi (IP address, connection state, battery percentage).
2. **Action Dashboard (Main Pane):** Interactive menu items you can navigate with arrow keys or vim keybindings (`j`/`k`):
    - **[M]irror Screen** (Launches `scrcpy` subprocess)

- **[S]creenshot** (Saves to Mac Desktop and shows a success toast)
- **[W]ireless Toggle** (Switches USB device to TCP/IP mode automatically)
- **[F]ocus Mode** (Mirrors screen while turning off the physical phone display)

3. **Live Log Stream (Bottom Pane):** A scrolling status window showing output logs from underlying `adb` commands (e.g., `adb connect successful`, `screenshot saved`).

Phase 3: Minimal Go + Bubble Tea TUI Blueprint

Below is a clean, modular example using **Go** and **Bubble Tea** to get your TUI skeleton up and running quickly.

1. Project Structure

Keep your code clean and adhere to file-size discipline by separating your TUI logic from your ADB/Scrcpy wrapper logic:

- `main.go` (Entry point)
- `tui/model.go` (Bubble Tea UI state and rendering)
- `bridge/adb.go` (Executes underlying system commands)

2. `tui/model.go` Example

```
package tui

import (
	"fmt"
	tea "github.com/charmbracelet/bubbletea"
)

type model struct {
	choices  []string
	cursor   int
	selected string
	status   string
}

func InitialModel() model {
	return model{
		choices:  []string{"Mirror Screen (Scrcpy)", "Take Screenshot", "Switch to Wireless (TCP/IP)", "Quit"},
		cursor:   0,
		selected: "",
		status:   "Ready. Select an option using arrows or j/k.",
	}
}

func (m model) Init() tea.Cmd {
	return nil
}

func (m model) Update(msg tea.Msg) (tea.Model, tea.Cmd) {
	switch msg := msg.(type) {
	case tea.KeyMsg:
		switch msg.String() {
		case "ctrl+c", "q":
			return m, tea.Quit

		case "up", "k":
			if m.cursor > 0 {
				m.cursor--
			}

		case "down", "j":
			if m.cursor < len(m.choices)-1 {
				m.cursor++
			}

		case "enter":
			m.selected = m.choices[m.cursor]
			if m.selected == "Quit" {
				return m, tea.Quit
			}
			m.status = fmt.Sprintf("Executing: %s...", m.selected)
			// Trigger your underlying adb/scrcpy function here based on m.selected
		}
	}
	return m, nil
}

func (m model) View() string {
	s := "📱 Android-Mac Bridge Dashboard\n\n"
```

```
	for i, choice := range m.choices {
		cursor := " "
		if m.cursor == i {
			cursor = ">" // Selection indicator
		}
		s += fmt.Sprintf("%s [%s]\n", cursor, choice)
	}

	s += fmt.Sprintf("\nStatus: %s\n", m.status)
	s += "\n(Press q to exit)"
	return s
}
```

Phase 4: Key Pro Tips for Your TUI Tool

- **Subprocess Management:** When launching `scrcpy` from your TUI, run it as an asynchronous background process so your terminal interface doesn't freeze or lock up while the mirror window is open.
- **Auto-Refresh:** Add a background tick command (`tea.Tick`) every 3 seconds to ping `adb devices` so your TUI updates instantly if you plug or unplug your phone cable.

User prompt: can i recors screen as vixeo as well

Response: Yes, you can record your Android screen as a video directly from your Mac CLI tool.

The easiest and most reliable way to achieve this without building complex video encoding pipelines is to leverage the built-in video recording feature of **ADB** or the streaming capabilities of **Scrcpy**.

Method 1: Using ADB (Saves directly to your phone storage, then pulls to Mac)

Android has a native screen recording utility built directly into its system shell via `screenrecord`. You can trigger this command from your script or CLI tool:

1. **Start Recording:**

   ```
   adb shell screenrecord /sdcard/capture.mp4
   ```

   (Note: ADB records until you stop the process with `Ctrl+C`, or it hits the maximum limit of 3 minutes).

2. **Pull the Video to your Mac:**

   ```
   adb pull /sdcard/capture.mp4 ~/Desktop/android_recording.mp4
   ```

3. **Clean up the phone storage:**

   ```
   adb shell rm /sdcard/capture.mp4
   ```

Method 2: Using Scrcpy (Streams directly to your Mac in real-time)

If you want to record the video file directly onto your Mac while viewing it live, modern versions of `scrcpy` have a built-in recording flag.

- **Run the mirror with recording enabled:**

  ```
  scrcpy --record ~/Desktop/scrcpy_recording.mp4
  ```

  This command will open the live mirroring window on your Mac and save an MP4 file of everything that happens on your screen. When you close the Scrcpy window, the recording is finalized and saved automatically.

Implementation Tip for Your CLI / TUI Tool

If you are implementing this into your tool (in Go or Python), you can wrap these commands into a simple background subprocess handler so your TUI doesn't lock up while recording:

- For ADB recording, start the process asynchronously, wait for user input (or a timer), send a `SIGINT` to stop `screenrecord`, and then execute the `adb pull` command.

User prompt: why the 3 min limit?

Response: The **3-minute limit** (`180 seconds`) is a hardcoded safety and buffer safeguard built directly into Android's native screen recording utility (`adb shell screenrecord`).

Android imposes this restriction for two primary technical reasons:

1. **RAM and Buffer Overflow Protection:** The `screenrecord` command encodes raw video frames on-the-fly using the device's hardware video encoder (MediaCodec) and buffers the output stream in device memory or temporary storage before writing it out. Unbounded recording on mobile hardware risks filling up available volatile memory or crashing the media server daemon if the storage write speed lags behind.
2. **Thermal Management:** Continuous hardware video encoding consumes significant CPU and GPU power, generating heat. Limiting native CLI recordings prevents accidental background runaway processes from overheating the phone battery or degrading device performance.

How to Bypass or Extend the Limit

If you are building your custom macOS CLI tool, you can easily override or extend this limit using command flags:

- **Using ADB:** You can explicitly pass the `--time-limit` flag to set a custom duration in seconds (up to a maximum of 3 minutes per single continuous clip, e.g., `--time-limit 120`):

  ```
  adb shell screenrecord --time-limit 120 /sdcard/capture.mp4
  ```

- **Using Scrcpy:** If you record via `scrcpy`, it streams and saves the video directly to your Mac storage in real-time, completely bypassing Android's internal 3-minute file limit. It will record continuously until you manually stop the process or close the window.

User prompt: i like the one without limits, will this interffear with drmc or otehr video players on the phone?

Response: Building your tool on top of raw stream capture and mirroring engines means **DRM (Digital Rights Management)** apps will behave predictably: **they will block the feed.**

How DRM Interacts with Your Tool

When you open streaming or banking apps that enforce DRM or secure window flags (`FLAG_SECURE`)—such as **Netflix, Disney+, Amazon Prime Video, banking apps, or password managers**—the app instructs Android's hardware composer to hide its contents from external capture paths.

- **The Result:** The moment you navigate into a protected video player or sensitive screen, the remote desktop window on your Mac will turn into a **solid black rectangle** (though audio might still play, or the app will block playback entirely).
- **Other Video Players:** Standard video players that do not use strict hardware DRM (like **VLC, local media players, YouTube, or open web video**) will mirror and record smoothly without any blackouts.

Why This Happens

This restriction is enforced at the Android OS and hardware level to prevent piracy and unauthorized capture of copyrighted media. Because tools like `scrcpy` and ADB rely on system-level mirroring hooks, they respect these security flags and cannot natively bypass them on standard unrooted devices.

Does it break your phone or other players?

- **No.** It will not damage your phone, corrupt your system, or interfere with normal video playback directly on the device itself.
- Normal video apps and local media files will play, mirror, and record normally on your Mac. Only apps explicitly protected by DRM security layers will trigger the black screen behavior on the Mac end.

Sources

1. DRM Black Screen When Screen Recording: Why It Happens

User prompt: lets say we build the too to use that to view phone screen on mac, lie remote desktop, if i watch a drm video on ntflix or vimio or even open a pnetime image on whatsapp will the screen be blank when viwing on mac as remote desktop usig scrappy

Response: Yes, it will go blank for specific apps, but how it behaves depends entirely on which app you are opening:

1. DRM Video Apps (Netflix, Disney+, Prime Video)

- **What happens:** The moment you play a video, the screen will instantly turn into a **solid black rectangle** on your Mac window.
- **Why:** These apps enforce Android's `FLAG_SECURE` window setting. Because `scrcpy` captures the display surface through the system's hardware compositor, the Android OS intentionally blocks the stream from rendering copyrighted content to external display tools.

2. Standard Web Video (Vimeo, YouTube browser/app)

- **What happens:** They **will mirror and play smoothly** without going blank.
- **Why:** Standard video players do not enforce rigid system-level DRM display blocks for basic playback, so `scrcpy` can capture and stream them normally.

3. Messaging Apps (Opening a preview image on WhatsApp)

- **What happens:** Regular chats and typing will mirror fine, but if WhatsApp enforces privacy settings (such as "View Once" media or secure chat overlays), or if you are looking at sensitive screens like banking details or password inputs, those specific windows will blank out. However, standard shared images viewed normally inside a standard chat thread will generally display fine unless the app or system triggers a security flag.

Sources

1. Flag_Secure Doesn't Stop Everything: Screen Share Bypass ... - PTKD

User prompt: ok, so waht are all the features we could includ in our tui app?

Response: A comprehensive feature roadmap for your custom macOS-to-Android TUI bridge tool turns it into a powerful, all-in-one developer and power-user dashboard. Grouping these capabilities into logical categories makes for a clean, intuitive menu layout.

1. Core Display & Media Control

- **Live Screen Mirroring (`scrcpy` integration):** High-performance, low-latency window streaming your phone screen onto your Mac.

- **Borderless / Frameless Window Toggle:** Strip away the Android system decoration window to make it float seamlessly on your macOS desktop.
- **Video Recording:** Record the live mirror stream directly to your Mac as an MP4 file with zero time limits.
- **Instant Screenshots:** Capture exact frames instantly and save them directly to your Mac desktop with a success toast notification.
- **Focus / Presentation Mode:** Mirror your phone while automatically turning off the physical phone screen to preserve battery during long casting sessions.

2. Connectivity & Device Management

- **Smart Connection Switcher:** Seamlessly toggle between a wired USB connection and a wireless TCP/IP connection with a single keystroke.
- **Multi-Device Selector:** If you have multiple Android devices plugged in or connected via Wi-Fi, easily switch focus between them from a sidebar device list.
- **Live Status HUD:** Real-time monitoring of device vitals like battery percentage, charging state, IP address, and connection type (USB vs. Wi-Fi).

3. File & Clipboard Utilities

- **Clipboard Sync:** Instantly push your Mac's clipboard text to your Android phone (or pull the phone clipboard back to your Mac).
- **Drag-and-Drop Sideloading:** Drag an APK file onto the TUI dashboard to install it instantly on the connected device.
- **Quick File Pull/Push:** Browse and transfer files between specific Android directories and your Mac via a simple file-transfer menu.

4. Hardware & Sensor Emulation / Control

- **Audio Forwarding:** Stream phone audio directly to your Mac speakers (supported natively by modern `scrcpy` backends).
- **Virtual Webcam:** Turn your phone's high-resolution rear or front camera into a Mac webcam over USB or Wi-Fi for video calls.
- **Device Control Shortcuts:** Quick hotkeys or menu items to lock the screen, trigger home/back buttons, adjust volume, or bring down the notification shade.

5. Debugging & Developer Tools

- **Live Logcat Stream:** A scrolling bottom pane in your TUI showing filtered system logs (`logcat`) or error crashes in real time.
- **App Management:** Quick options to force-stop running packages, clear app data, or uninstall user packages without digging through phone settings.
- **Shell Command Runner:** A built-in terminal input box inside your TUI to execute quick custom `adb shell` commands on the fly.

User prompt: waht other features can we add

Response: To elevate your macOS-to-Android CLI and TUI bridge tool from a standard screen mirror utility into an elite power-user suite, you can integrate several advanced, highly practical features.

1. Advanced Device & System Management

- **App Management Dashboard:** A dedicated view to list installed packages (`pm list packages`), search for apps, install/sideload APKs with a drag-and-drop prompt, or instantly clear app data and cache without touching the phone.
- **Network & Port Forwarding Manager:** Easily set up reverse port forwarding (`adb reverse tcp:8080 tcp:8080`) or local port forwarding so you can test local web apps running on your Mac directly inside your mobile browser.
- **File Explorer & Quick Transfer:** A simple dual-pane or pick-list file browser to pull files from standard directories (`/sdcard/Download/`) straight to your Mac, or push files onto the device.

2. Audio & Multimedia Control

- **System Audio Forwarding:** Route your phone's internal audio playback straight to your Mac speakers using modern `scrcpy` audio flags, so media and notifications play on your computer.
- **Microphone Passthrough:** Use your Mac's connected microphone as the audio input for voice calls or voice notes on your Android device.
- **Virtual Webcam Stream:** Turn your phone's high-res camera into a plug-and-play webcam for your Mac over USB or Wi-Fi for video calls or streaming.

3. Productivity & Automation Shortcuts

- **Bi-directional Clipboard Sync:** Automatically keep your Mac and Android clipboards in sync, or provide explicit push/pull commands in the TUI menu.
- **Batch Macro Runner:** Execute predefined sequences of ADB commands (e.g., tap at coordinates `X, Y`, wait 2 seconds, input text, press home) to automate repetitive testing tasks.
- **Custom Intent Launcher:** Fire specific Android intents from your terminal or TUI to instantly open a specific app screen, deep link, or URL.

4. Diagnostics & Developer Metrics

- **Live Logcat Viewer with Filters:** A dedicated log streaming pane inside your TUI that lets you filter logs by tag, priority level (Error, Warning, Info), or search keywords in real time.
- **Performance HUD (CPU / RAM Overlay):** Pull live system resource usage (`dumpsys cpuinfo` and `meminfo`) to monitor how apps are performing on the device.

- **Device Vitals & Health Check:** Real-time diagnostics tracking battery temperature, charging health, storage capacity, and screen resolution settings.

User prompt: how about storage management like my diskman tool that i have build for mac, can we do the same features using the cli bright to see and manage phones storage, you can see my diskman tool on my github

Response: Yes, absolutely. You can bring the exact disk-inspection, file-sorting, and space-cleanup concepts from your **DiskMan / GoDisk** tools directly to your Android bridge CLI, tailoring them to analyze phone storage over ADB.

While macOS file systems are accessed via standard directory traversal, Android uses its Linux-based internal storage and virtual file systems (like `/sdcard` and `/storage/emulated/0`), meaning your engine will issue ADB shell commands and parse the output rather than reading local disk sectors directly.

Core Storage Features to Port from DiskMan to Android

1. **Interactive Storage Tree & Size Breakdown (`diskman-droid scan`)**
   - *How it works:* Instead of running `du -sh *` on a local Mac directory, your CLI executes `adb shell du -h --max-depth=1 /sdcard/` concurrently or sequentially, parses the byte sizes, and renders them in a ranked breakdown or an interactive tree view using Bubble Tea.
   - *What it surfaces:* Instantly shows whether your `WhatsApp/Media`, `DCIM/Camera`, or `Android/obb/` directory is eating up your phone's storage.
2. **Large File & Cache Hunter (`diskman-droid clean`)**
   - *How it works:* Scans directories for files exceeding a specified size threshold (e.g., >100MB) or targets known application cache paths (`/sdcard/Android/data/*/cache/`).
   - *Actionable cleanup:* Lets you select and delete or pull-then-delete bulky files directly from your terminal interface without manually digging through the Android Settings app.
3. **Duplicate File Finder via Hashing (`diskman-droid dupes`)**
   - *How it works:* Pulls file metadata or runs a lightweight script via ADB shell to compute checksums (MD5/SHA256) of media files, flagging duplicate photos or downloaded documents across internal storage.
4. **App Storage Footprint Inspector (`diskman-droid apps`)**
   - *How it works:* Queries Android package stats (`adb shell pm path` combined with package size dumpsys) to list installed apps sorted by total footprint (Code size + Data size + Cache size), letting you clear app data or cache with a single keystroke.

Implementation Pattern (Go + ADB)

Since you already have experience building CLI utilities in Go with Bubble Tea, querying Android storage follows a clean subprocess execution pattern. For example, fetching and sorting top-level folder sizes on the device:

```
package bridge

import (
	"bytes"
	"os/exec"
	"strings"
)

// FolderSize holds directory path and formatted size
type FolderSize struct {
	Path string
	Size string
}

func GetStorageSummary() ([]FolderSize, error) {
	// Execute du command inside android shell
	cmd := exec.Command("adb", "shell", "du", "-h", "--max-depth=1", "/sdcard/")
	var out bytes.Buffer
	cmd.Stdout = &out

	err := cmd.Run()
	if err != nil {
		return nil, err
	}

	var results []FolderSize
	lines := strings.Split(out.String(), "\n")
	for _, line := range lines {
		parts := strings.Fields(line)
		if len(parts) >= 2 {
			results = append(results, FolderSize{
				Size: parts[0],
				Path: parts[1],
			})
		}
	}
	return results, nil
}
```

By integrating this into your TUI dashboard, you can add a dedicated **"Storage Analyzer"** tab that gives you the exact same visibility and cleanup power over your mobile device that DiskMan provides for your Mac.

User prompt: not give me to full tool set we plan in this app

Response: 📱 Android-Mac Bridge & Storage TUI Dashboard

A complete architectural and feature roadmap for your custom macOS CLI and Bubble Tea TUI bridge tool is outlined below, combining low-level ADB and Scrcpy automation with the advanced disk and file analysis concepts from your DiskMan tools.

Phase 1: Core Command-Line Engine & Subprocess Architecture

Your CLI tool runs as a single compiled binary (written in Go) that orchestrates underlying system binaries (`adb` and `scrcpy`) asynchronously.

- **Dependency Bootstrap:** Automatically verifies if `platform-tools` and `scrcpy` are present on your Mac, offering a quick install prompt via Homebrew if missing.
- **Connection Multiplexer:** Automatically detects connected devices via USB or wireless TCP/IP, tracking device states, battery health, IP addresses, and active modes.

Phase 2: Comprehensive TUI Feature Breakdown

Your Bubble Tea TUI interface is organized into interactive tabs and navigation views:

1. Screen Mirroring & Display Suite

- **Low-Latency Mirroring (`scrcpy`):** High-performance live window streaming with customizable resolution, bitrate, and framerate options.
- **Borderless / Float Mode:** Strips away window decorations so your mobile screen floats seamlessly on your Mac desktop.
- **Focus / Presentation Mode:** Mirrors the display while automatically turning off the physical phone screen to conserve battery during long sessions.
- **App Privacy / DRM Handling:** Gracefully handles secure windows (automatically showing blackouts for protected banking or DRM video apps like Netflix while maintaining smooth streaming for standard media).

2. Capture & Recording Suite

- **Instant Screenshots:** Grabs exact frames instantly and saves them directly to your Mac desktop with success toasts.
- **Unlimited Video Recording:** Records live video streams directly to your Mac storage with zero time limits, bypassing Android's native 3-minute file restrictions.
- **Clipboard Synchronization:** Instantly pushes your Mac clipboard text to your Android device or pulls clipboard strings back to your Mac.

3. Mobile Storage Inspector (Ported from DiskMan)

- **Interactive Storage Tree (`droid scan`):** Scans internal storage (`/sdcard/`) concurrently via ADB to build a ranked breakdown of top-level folder sizes (identifying heavy directories like `WhatsApp/Media`, `DCIM/Camera`, or `Android/obb/`).
- **Large File & Cache Hunter (`droid clean`):** Filters files exceeding custom size thresholds (e.g., greater than 100MB) or targets hidden application cache paths, letting you select and clear bulky files directly from your terminal.
- **App Footprint Analyzer:** Queries Android package stats (`pm` and `dumpsys`) to list installed applications sorted by total storage usage (Code + Data + Cache size), allowing you to clear app data with a single keystroke.

4. Advanced Hardware & Developer Tools

- **System Audio & Mic Routing:** Forwards device audio playback straight to your Mac speakers or uses your Mac microphone as the phone audio input.
- **Virtual Webcam:** Turns your phone's high-res camera into a plug-and-play webcam for your Mac over USB or Wi-Fi.
- **Live Logcat Stream:** A scrolling bottom pane in your TUI showing filtered system logs (`logcat`) or error crashes in real time.
- **App Sideloading & Management:** Drag-and-drop APK sideloading, package uninstallation, and force-stopping misbehaving apps directly from the dashboard.

User prompt: we need a nice name

Response: Here are some cool, punchy name ideas for your Android-Mac bridge and storage TUI tool, categorized by the vibe they give off:

⚡ Sleek & Minimalist

- **BridgeOS** — Clean, professional, and implies a seamless connection between two operating systems.
- **Linker** — Short and sharp; focuses on the connection bridge.
- **WireDroid** — Simple nod to the USB and wireless flexibility of the tool.
- **DroidLink** — Direct, descriptive, and easy to remember.

🛠️ Developer & CLI Vibe

- **Adbmux** — Combines ADB with a multiplexer/dashboard feel.
- **TermDroid** — Emphasizes the terminal and TUI-first nature of the app.
- **ShellBridge** — Sounds like a robust, low-level engineering tool.
- **DroidX** — Sleek extension of your device bridge toolset.

📱 Ecosystem & Flow

- **MacDroid** — Clear and straightforward about what systems are meeting.
- **NexusCLI** — Gives it a central hub or control-center feel.
- **FlowDroid** — Focuses on the smooth, lag-free mirroring and file transfer.
- **OmniDroid** — Implies full control over storage, screens, and device stats in one place.

Do any of these hit the mark, or are you leaning toward a specific style?

User prompt: i like DroidLink and DroidBridge and even DroidWire, can you check if theres any similar name app are there in the market so we dont have to deal with trademarks

Response: Checking the market shows that all three names have some degree of overlap or prior usage in the mobile ecosystem, which is important to keep in mind regarding trademarks or namespace clarity:

- **DroidLink:** There are existing Android apps on Google Play using variants of this name (typically focused on simple remote control or deep-linking generation).
- **DroidBridge:** There are active developer utilities and community launchers (such as tools used for running specific environments like Minecraft Java on Android) registered under similar names.
- **DroidWire / WireDroid:** "WireDroid" is taken by an embedded circuit workspace app on the Play Store, though "DroidWire" has less direct friction.

Safer & More Unique Alternatives

To completely steer clear of existing utility trademarks while keeping that clean, power-user feel, you could pivot slightly to something distinct:

- **TermDroid** or **DroidMUX** (leans into the CLI and multiplexer architecture)
- **MacBridge** or **BridgeCLI** (focuses on the connection bridge rather than just the OS prefix)
- **AdbFlow** or **ScrcpyGo** (describes the underlying stack neatly)

If you publish this as an open-source tool on GitHub or distribute it independently as a CLI binary, exact trademark enforcement is less aggressive than a formal commercial app store filing, but choosing a unique name keeps your GitHub repository and binary name clean.

User prompt: lets go with TermDriod for now abd i forgot to ask is using scrcpy ok to build out own tool that we plan to publish on github? will i get in to any drouble

Response: **TermDroid** is an awesome name. It sounds crisp, developer-focused, and fits right in with terminal tooling.

Regarding publishing your tool on GitHub and using `scrcpy` under the hood: **you will not get into trouble, provided you follow a few simple guidelines.**

1. The Licensing Reality

`scrcpy` is licensed under the **Apache 2.0 License**. This is a very permissive open-source license. It explicitly grants you permission to:

- **Commercial use:** Use and distribute the software freely, even for commercial projects.
- **Modification:** Alter or wrap the code.
- **Distribution:** Include or bundle it as part of your own software packages.

2. How to Stay Clear of Trouble on GitHub

Because you are building a wrapper CLI/TUI tool that *orchestrates* `scrcpy` (i.e., your tool executes the `scrcpy` binary via system subshells rather than modifying or copying its source code line-by-line), you are creating what Apache 2.0 considers a **"Larger Work"** or a separate frontend component.

To keep everything clean and compliant:

- **Include the License Notice:** In your GitHub repository, include a standard `LICENSE` file for TermDroid, and add a quick attribution note in your README or a `NOTICE` file acknowledging that your tool relies on `scrcpy` and `adb` as underlying binaries.
- **Don't Claim Ownership of the Engine:** Just make it clear in your documentation that TermDroid is an orchestrator/TUI layer, and that users need `scrcpy` and `platform-tools` installed (or your tool can automatically check and fetch them via Homebrew).
- **Avoid Bundling Binaries Directly (Optional best practice):** Instead of uploading compiled `scrcpy` or `adb` binaries directly inside your own git repository, let your CLI check for them or prompt the user to install them via Homebrew (`brew install scrcpy android-platform-tools`). This keeps your repo lightweight and completely decouples your code from third-party binaries.

You are completely safe to build, open-source, and share TermDroid on GitHub.

User prompt: and can we have more features to make the scrcpy only a minor part of our tool and most of the other features use direct adb commands to prevent this seam like just a scrcpy wrapper only

Response: That is a brilliant architectural pivot. By leaning heavily into **direct ADB shell commands, raw socket tunnels, and Linux system utilities** on the device side, **TermDroid** becomes a true all-in-one developer and power-user suite where `scrcpy` is merely one tool in a massive utility belt.

Shifting the core gravity away from screen mirroring lets us stack a massive array of native ADB and device-control features into the TUI.

🚀 Expanded Native Feature Set (Beyond Scrcpy)

1. Advanced Device & Process Control (`termroot` / `termproc`)

- **Process Monitor & Killer:** Live list of running Android processes (parsed via `adb shell ps -A`), allowing you to sort by CPU/RAM usage and force-kill rogue background tasks instantly.
- **Activity & Intent Launcher:** Fire explicit or implicit Android intents straight from your terminal (e.g., launching a deep link or opening a specific settings pane instantly).
- **Keyevent Injector:** Remotely simulate hardware inputs (`adb shell input keyevent`), inject custom text strings, or trigger touch taps/swipes at exact screen coordinates.
- **Volume & Brightness Tweak:** Fine-tune device audio volume levels and screen brightness without touching the physical rocker buttons.

2. Deep Storage & File Management (Your DiskMan Engine for Android)

- **Interactive Storage Tree Scan (`termdroid storage scan`):** Walks `/sdcard/` concurrently via ADB to build a high-performance hierarchical breakdown of folder sizes, letting you pinpoint what is consuming your mobile storage.
- **Bulky File & Cache Purger (`termdroid storage clean`):** Scans for cached application data (`/sdcard/Android/data/*/cache/`) and large files over user-defined thresholds (e.g., >50MB) with one-click deletion.
- **App Footprint Analyzer:** Evaluates APK installation paths, data sizes, and internal cache footprints, offering instant clearing of app data or cache.
- **Wireless File Transfer Hub:** Quick bi-directional file push/pull manager with progress bars, letting you grab logs, photos, or documents over Wi-Fi or USB.

3. Real-Time Diagnostics & System Telemetry (`termdroid health`)

- **Live Logcat Streamer:** A dedicated, filterable log viewer with regex search and priority toggles (Verbose, Debug, Info, Warning, Error, Fatal) running directly inside a TUI pane.
- **Battery & Power Vitals HUD:** Real-time tracking of battery temperature, charging current/voltage, health status, and estimated mAh capacity.
- **Network & Port Forwarding Manager:** Instantly set up local port forwarding (`adb forward`) or reverse port forwarding (`adb reverse`) to tunnel local web development servers from your Mac straight into your phone's browser.
- **Package Manager Inspector:** List all installed packages (`pm list packages`), filter by third-party vs. system apps, inspect package manifests, or perform batch uninstalls/sideloads.

4. Hardware & Automation Utilities (`termdroid utils`)

- **Clipboard Sync Bridge:** Automatically poll the Android clipboard or push your Mac's clipboard contents back and forth over ADB.
- **Quick Wi-Fi Tether / TCP/IP Switcher:** Automatically handle the transition from a wired USB debugging session to a wireless TCP/IP connection on a custom port.
- **Custom Macro / Script Runner:** Execute sequences of ADB commands (e.g., tap, wait, input text, take screenshot) stored in local configuration files to automate repetitive testing routines.

Architectural Design: Why this matters

By writing these modules in Go using direct `os/exec` wrappers around `adb`, your tool operates as a robust, standalone system utility. `scrcpy` is simply isolated to a single function handler (`func StartMirror()`), proving that TermDroid is an independent device management suite rather than a thin wrapper.
